Yenepoya (Deemed to be University).

Project – Low Level Design

on

Zero-Day Scanner for Supply Chain Web Portals

Student Name:

Ashin Mohammed KT

Industry Mentor: Mr. Shashank Korram **Guided by: Mr. Vikyath**

Table of Contents

1. Introduction 3
   1.1 Scope of the document 3
   1.2 Intended Audience 3
   1.3 System overview 3

2. System Design 4
   2.1. Application Design 4
   2.2. Process Flow 4
   2.3. Information Flow 5
   2.4. Components Design 6
   2.5. Key Design Considerations 7
   2.6. API Catalogue 7

3. Data Design 8
   3.1. Data Model 8
   3.2. Data Access Mechanism 8
   3.3. Data Retention Policies 9
   3.4. Data Migration 9

4. Interfaces 10

5. State and Session Management 11

6. Caching 11

7. Non-Functional Requirements 11
   5.1. SecurityAspects 11
   5.2 Performance Aspects 11

8. References 12

Low-Level Design (LLD):Zero-Day Scanner for Supply Chain Web Portals

1. INTRODUCTION

1.1. SCOPE OF THE DOCUMENT

This Low-Level Design (LLD) document provides a comprehensive and detailed technical specification of the project titled **"Zero-Day Scanner for Supply Chain Web Portals."**

The purpose of this document is to describe the internal working, system architecture, and implementation-level details of the application. It explains how different components interact, how data flows through the system

The document covers the following aspects:

- Component-level design
- Data flow and process flow
- API structure and routing
- Data handling and in-memory processing
- User interface and system interactions
- Security and performance considerations

This project simulates real-world **ethical hacking workflows**, focusing on identifying:

- Broken links (error pages) , Redirect and misconfigured endpoints, Potentially risky external links

1.2. INTENDED AUDIENCE

This document is intended for the following readers:

1. Students learning ethical hacking, secure development, and web testing
2. Faculty members evaluating the project design and implementation quality
3. Cybersecurity trainees who want to understand how crawling and automated link inspection work in a manufacturing context

1.3. SYSTEM OVERVIEW

The system is a web-based application where users enter a URL of a supply chain portal to analyze its links. The backend processes the request and performs automated link scanning to check the validity and safety of hyperlinks.

The system simulates basic security scanning by identifying working, broken, and potentially risky links, helping ensure reliability in supply chain web applications.

Key Functionalities

- **Web Crawling:** Collects all links from the given webpage
- **Link Extraction:** Extracts <a href=""> elements using HTML parsing
- **URL Normalization:** Converts relative links into absolute URLs
- **Status Code Checking:** Verifies each link using HTTP requests
- **Link Categorization:**
  - Working (200)
  - Broken (4xx, 5xx)
  - Redirects (3xx)
  - External links

2. SYSTEM DESIGN

2.1. APPLICATION DESIGN

The Zero-Day Scanner follows a **Synchronous Request–Response architectural pattern**, where the client submits a URL through a form using a POST request, and the Flask server processes the entire scanning operation before returning a complete HTML results page. The system does not use real-time streaming or event-driven mechanisms, making it simple and suitable for small-scale applications.

The application is designed using a **monolithic, single-tier architecture**, where all backend functionalities such as routing, business logic, link extraction, and status checking are implemented within a single file (app.py). This approach ensures simplicity, easy debugging, and faster development.

The system is structured into multiple logical layers. The **presentation layer**, built using HTML5, CSS3, and basic JavaScript, is responsible for handling user input and displaying results. The **application layer**, implemented using Python and Flask, manages routing, request handling, and overall control flow of the program. The **network layer** uses the Requests library to send HTTP and HTTPS requests to fetch webpage content and validate links. The **parsing layer**, powered by BeautifulSoup, processes the HTML content and extracts all hyperlinks. Additionally, a **concurrency layer** using ThreadPoolExecutor is used to perform parallel HTTP requests, improving the performance of link checking.

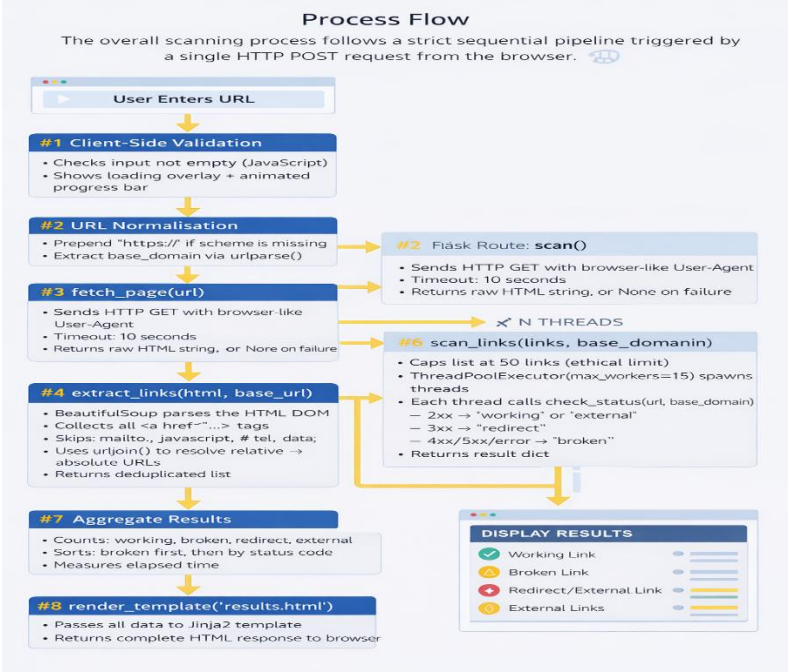
The project follows a simple and organized directory structure. The main application logic is contained in app.py, while dependencies are listed in requirements.txt. A separate test_app.py file is used for testing purposes. The static folder contains the CSS file for styling the user interface, and the templates folder contains HTML files, including index.html for the homepage and results.html for displaying the scanning results.

Overall, this design ensures that the system remains lightweight, easy to understand, and efficient for demonstrating web crawling and automated link scanning concepts in a supply chain context.

2.2. PROCESS FLOW

The execution follows a strict linear sequence:

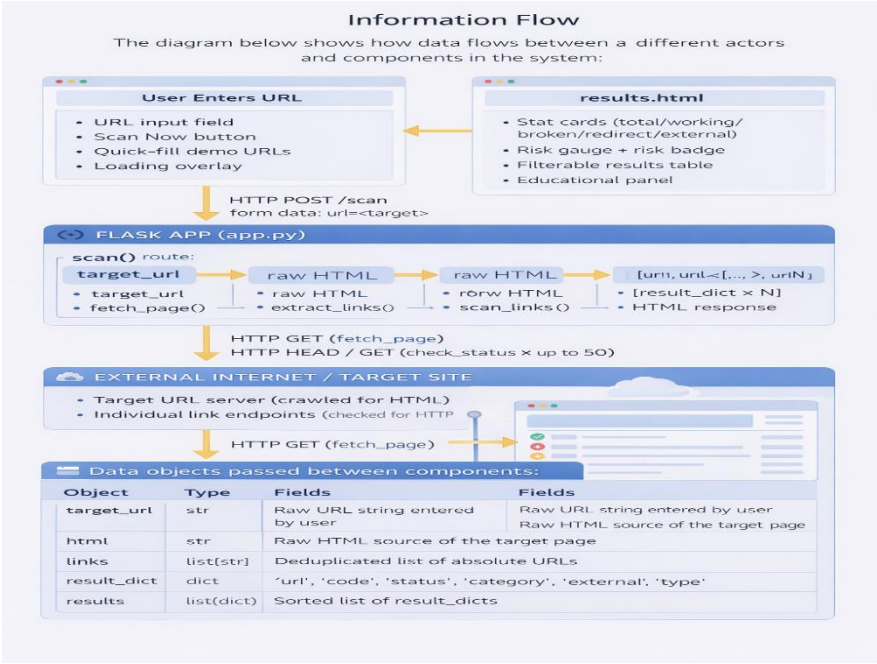
1. The user enters a URL in the input field on the homepage.
2. Client-side validation checks that the input is not empty and displays a loading indicator.
3. The browser sends an HTTP POST request to the Flask server (`/scan` route).
4. The server normalizes the URL by adding “https://” if missing and extracts the base domain.
5. The `fetch_page()` function sends an HTTP request to retrieve the webpage content.
6. The `extract_links()` function parses the HTML using BeautifulSoup and extracts all valid hyperlinks.
7. Relative URLs are converted into absolute URLs, and duplicate links are removed.
8. The system limits the number of links (e.g., 50) to ensure efficient and ethical scanning.
9. Multiple threads are created using `ThreadPoolExecutor` to check link statuses in parallel.
10. The `check_status()` function sends HTTP requests to each link and determines the status code.
11. Links are categorized into working, broken, redirect, or external based on their response.
12. The system aggregates the results, counts each category, and measures total scan time.
13. Finally, the results are passed to the `results.html` template and displayed to the user.



2.3. INFORMATION FLOW

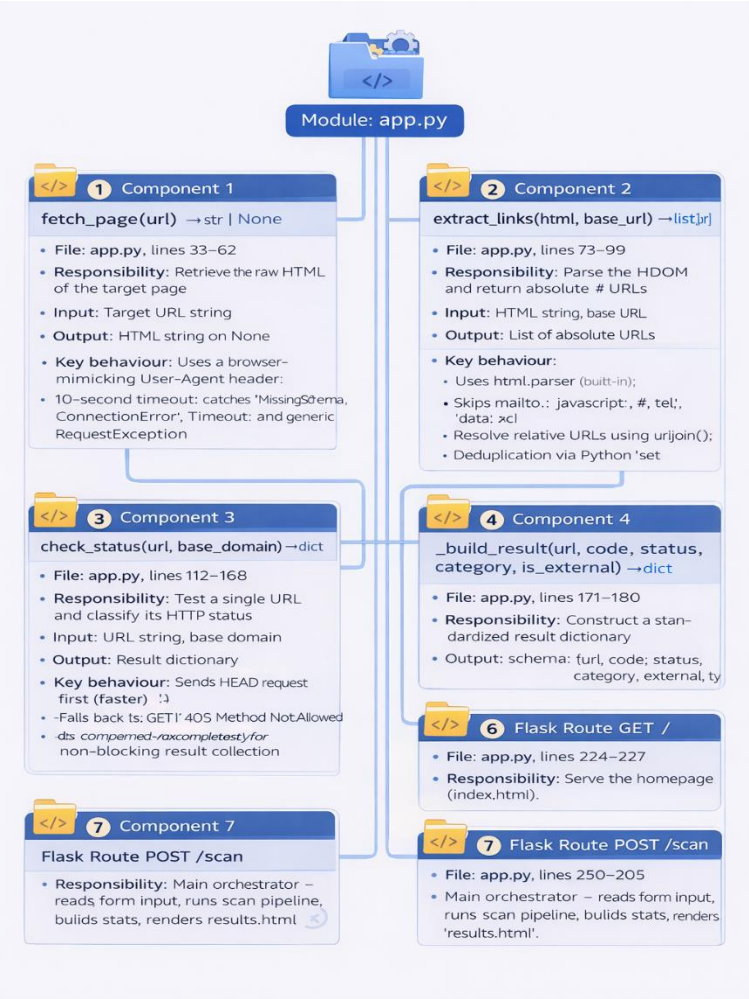
The user enters a URL in the browser through the input form and submits the request to the Flask server. The server receives the URL and fetches the webpage content using HTTP requests. The HTML is then parsed to extract all hyperlinks, which are converted into absolute URLs and filtered to remove duplicates. Each link is checked by sending HTTP requests to determine its status. The results are categorized into working, broken, redirect, and external links. Finally, the processed data is passed to the results page and displayed in the browser.

- Output Layer: The final output is a probability vector for classification and a Graphical Confusion Matrix that provides a visual representation of accuracy, highlighting the model errors.



2.4. COMPONENT DESIGN

- The system is divided into multiple components, each handling a specific task in the scanning process.
- The fetch_page() component is responsible for retrieving the HTML content of the target webpage.
- The extract_links() component parses the HTML and extracts all valid hyperlinks from the page.
- The check_status() component checks each link by sending HTTP requests and determines its status.
- The _build_result() component creates a structured result object for each link.
- The scan_links() component manages multiple link checks using parallel processing for better performance.
- The Flask route GET / handles loading the homepage interface.
- The Flask route POST /scan acts as the main controller, executing the full scanning process and returning results.



2.5. KEY DESIGN CONSIDERATIONS

- The system uses HTTP HEAD requests to check link status because they are faster and only fetch headers. However, if a server does not support HEAD requests, a GET request is used as a fallback to ensure compatibility.
- To maintain ethical scanning practices, the system limits the number of links to 50. This prevents excessive requests that could overload the target server or resemble a Denial of Service attack.
- The application uses 15 worker threads to perform parallel link checking. This provides a balance between faster performance and efficient use of system resources, avoiding issues like system overload or server rate limiting.
- The system follows a stateless design, meaning no database or session storage is used. Each request is processed independently, making the application simple, lightweight, and secure.
- The urljoin() function is used for URL normalization to correctly handle relative links and convert them into absolute URLs according to standard rules.
- A Python set is used to remove duplicate links efficiently, ensuring that each link is checked only once and improving performance.
- The system uses a browser-like User-Agent header while sending requests. This helps avoid blocking by servers that restrict requests from automated scripts.

2.6. API CATALOGUE

The application exposes two HTTP endpoints:

API Catalogue

API Catalogue

Endpoint	Method	Path	Description	Request Details	Response	Handler
Homepage	GET	/	Serves the main page with URL input form	No request body	HTTP 200 – Returns HTML page	index()
Scan URL	POST	/scan	Accepts URL, performs crawling and link scanning	HTTP 200 – templates/index .html	HTTP 200 – – Returns HTML with results or error message	

Additional Details (for /scan endpoint)

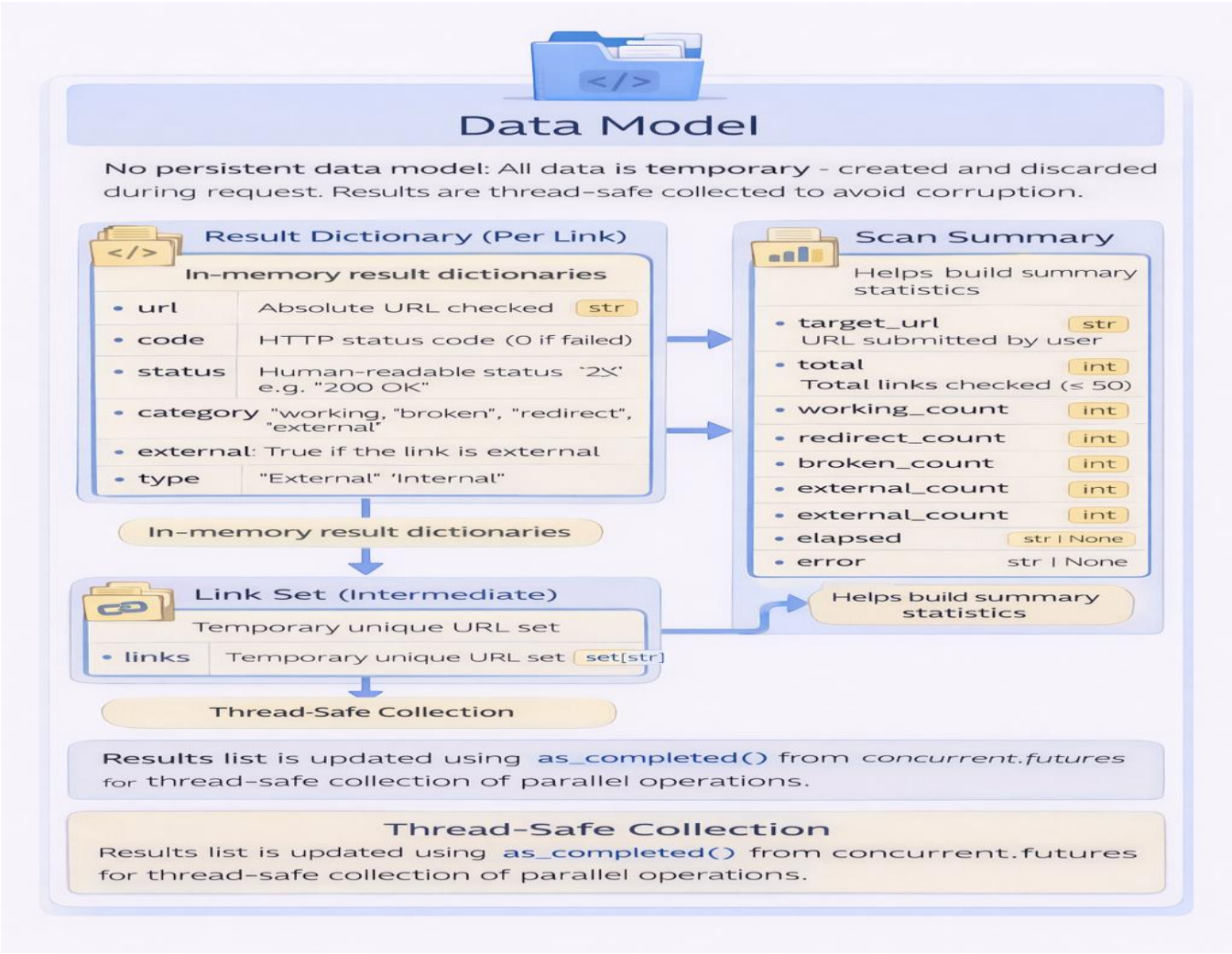
Field	Value	Response	Template
Template Variables	target_url, results, total, working_count, broken, count, redirect, count, external, count, elapsed,	HTTP 200 – Returns HTML with results or	templates/index.html
		HTTP 200 – Returns HTML with results or error message	templates/results.html

Additional Details (for /scan endpoint)

Field	Value
Template Variables	target_url, 'results, total, 'working_count, broken_count, redirect_count, external_count, elapsed, error
Error Handling	Displays error message on results page if scan fails
Authentication	Not required
API Type	Traditional server-rendered (No REST/JSON APIs)

3.DATA DESIGN

3.1.DATA MODEL



3.2. DATA ACCESS MECHANISM

Since the application does not use a database, all data is handled in-memory during the request lifecycle. The raw HTML content is fetched using an HTTP request and stored as a string. Extracted links are parsed from the HTML and stored temporarily using Python data structures such as sets and lists.

Each link's result is processed and stored in a list, which is later used to compute summary statistics using simple list operations. The processed data is then passed to the frontend using Flask's render_template() function as keyword arguments.

Jinja2 templating is used to dynamically render the HTML page, allowing the results to be displayed in a structured and user-friendly format.

3.3. DATA RETENTION POLICIES

The application does not store any data permanently on disk during its operation. All data, including HTML content, extracted links, and scan results, are stored temporarily in memory and exist only within the scope of the scan() function. Once the request is completed, this data is automatically cleared by Python's garbage collection.
The system does not generate or store logs in files, as all outputs are displayed directly in the browser. Additionally, no cookies, sessions, or browser storage mechanisms are used to retain user data between requests.
The application follows a stateless design, meaning each scan request is processed independently without retaining any previous data, ensuring simplicity and improved privacy.

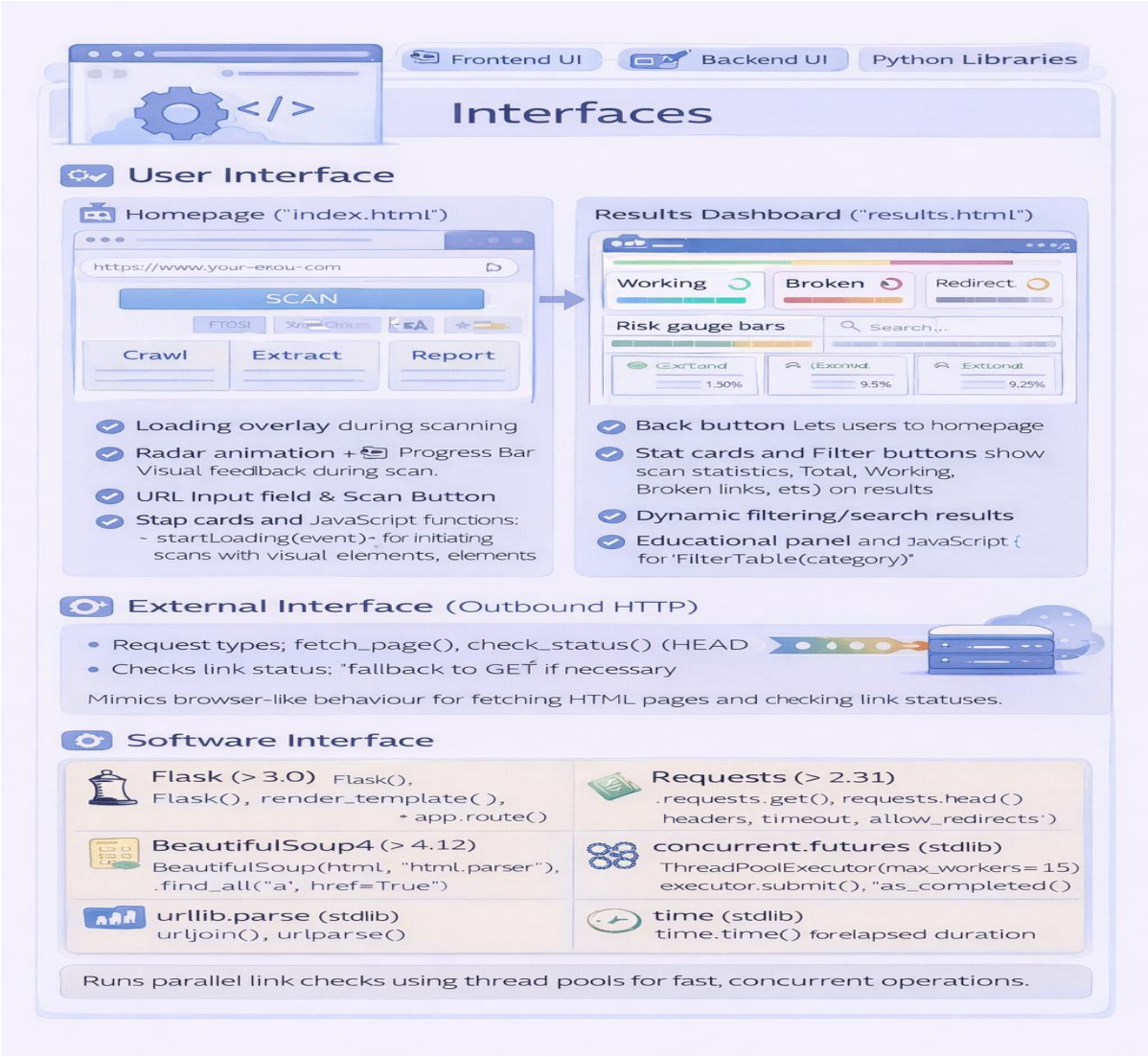
3.4. DATA MIGRATION

- Data migration is not applicable in this system, as the application does not use any database or persistent storage mechanism. There are no schema definitions, versioning, or migration processes involved.
- Since all data is processed temporarily in memory and discarded after each request, there is no requirement for data transformation or migration.
- However, in future enhancements, if a database is introduced to store scan history or logs, standard migration tools such as Flask-Migrate or Alembic can be used to manage schema changes. Currently, this is beyond the scope of the application.

4.INTERFACES

The system provides a user-friendly interface that allows users to interact with the application through a web browser. The homepage enables users to enter a URL, initiate the scan, and view loading animations that indicate the scanning process.

- The results dashboard displays the analyzed data in a structured format, including statistics, categorized links, and filtering options. It also provides search functionality and visual indicators such as risk levels to enhance user understanding.
- The application communicates with external web servers using HTTP requests to fetch webpage content and check link statuses. These outbound requests ensure that real-time data is collected directly from target websites. The system also interacts with various software libraries such as Flask, Requests, and BeautifulSoup to handle routing, network communication, and HTML parsing. These components work together to process data efficiently and display results dynamically using template rendering.



5.STATE AND SESSION MANAGEMENT

- The Zero-Day Scanner is a fully stateless application, meaning it does not maintain any server-side or client-side state between requests. The Flask server does not use sessions, cookies, or any form of persistent storage, and each request is processed independently.
- On the client side, no data is stored using cookies, localstorage, or sessionstorage. Temporary states such as loading indicators and filter selections are managed only within the browser using javascript during the current page lifecycle.
- All scan results exist only during the execution of a request as local variables in the backend and are discarded once the response is sent. As a result, if the user submits the same URL again, the system performs a fresh scan from scratch.
- This stateless design keeps the application simple, secure, and suitable for an educational use case.

6.CACHING

The Zero-Day Scanner does not implement any caching mechanism at any level of the system. The Flask server does not use server-side caching, and no caching libraries are configured. Each request is processed independently, and fresh data is fetched every time.

The application does not explicitly set HTTP cache headers, and since the scan operation uses a POST request, the browser does not cache the results page. All webpage content is retrieved in real-time using HTTP requests, ensuring that the latest data is always used for analysis.

Similarly, link status checks are performed freshly for each scan without storing previous results. This guarantees accuracy but may increase processing time for repeated scans.

7.NON-FUNCTIONAL REQUIREMENTS

5.1.SECURITY ASPECTS

The application includes basic security measures to ensure safe and ethical usage. It limits scans to 50 links to prevent misuse such as denial-of-service attacks. No user data, credentials, or scan results are stored, ensuring user privacy.

The system uses Jinja2 auto-escaping to prevent XSS attacks and avoids executing any user-provided code. External links are opened safely using rel="noopener" to prevent browser-based attacks.

Input URLs are validated and formatted properly before processing. The application also clearly identifies itself through a custom User-Agent for transparency. However, since this is an educational tool, protections against advanced threats like SSRF are not fully implemented, and debug mode is not intended for production use.

5.2. PERFORMANCE ASPECTS

- The application is optimized for performance using parallel processing, where multiple links are checked simultaneously using a thread pool. This significantly reduces the total scanning time compared to sequential execution.

To improve speed, the system primarily uses HTTP HEAD requests, which fetch only headers instead of full page content. Timeouts are set for both page fetching and link checking to prevent delays caused by slow or unresponsive servers.

The system limits scanning links to ensure predictable performance and avoid excessive processing time. Duplicate links are removed using efficient data structures, reducing unnecessary network calls.

8.REFERENCES

- Flask Documentation — https://flask.palletsprojects.com/en/3.0.x/
- Python Requests Library — https://docs.python-requests.org/en/latest/
- BeautifulSoup4 Documentation — https://www.crummy.com/software/BeautifulSoup/bs4/doc/
- Python concurrent.futures Module — https://docs.python.org/3/library/concurrent.futures.html
- MDN Web Docs: HTTP Status Codes — https://developer.mozilla.org/en-US/docs/Web/HTTP/Status
- RFC 3986: Uniform Resource Identifier (URI) — https://datatracker.ietf.org/doc/html/rfc3986
- OWASP: Broken Link Hijacking — https://owasp.org/www-community/attacks/Broken_Link_Hijacking
- Python urllib.parse Documentation — https://docs.python.org/3/library/urllib.parse.html
- Jinja2 Templating Engine — https://jinja.palletsprojects.com/en/3.1.x/